
Chapter 4 | 模型训练与量化导出

- 本章目标：给 NPU 提供权重和验证数据。
- 内容：tiny CNN、MNIST、后训练量化、导出格式。
- 产出：npu_mnist.py + npu_data/ 目录。

Chapter 4 | 为什么先做分类

- 分类模型只有 conv + pool + FC。
- 没有 YOLO 的检测头、NMS、多尺度。
- MNIST 小、训练快、验证直观。
- 跑通后可以替换成更大的数据集/模型。

Chapter 4 | 模型结构

我们选一个很小的 CNN：两层 3x3 卷积（8 通道、16 通道），中间夹 MaxPool，最后接 64 和 10 两个全连接。总参数约 5 万，int8 后约 50KB，NPU 片上 SRAM 放得下。

MNIST 是 28x28 灰度图，单通道。两层卷积后变成 7x7x16，展平 784 维，正好接 FC。结构简单但足以演示完整链路。

搭神经网络像搭乐高：先用卷积“看”局部特征，用池化“缩小”图片，用全连接“总结”成 10 个类别的分数。

我们的模型很小：两层卷积 + 两个全连接，约 5 万参数。小不是缺点——它刚好能塞进 NPU 的片上 SRAM，也足够演示“训练 -> 量化 -> 硬件推理”的完整流程。

- Conv(1->8, 3x3) + ReLU + MaxPool -> 14x14x8。
- Conv(8->16, 3x3) + ReLU + MaxPool -> 7x7x16。
- Flatten(784) -> FC(64) -> ReLU -> FC(10)。
- 参数约 5 万，int8 权重约 50KB。

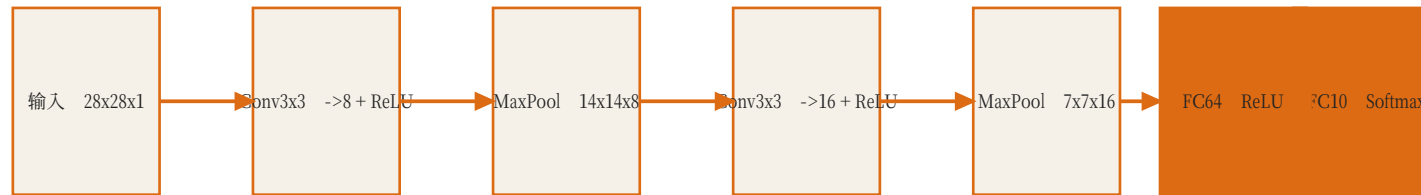


图 12 : tiny CNN = 卷积 + 池化 + 全连接, 没有检测头/NMS, 是 NPU 入门的最佳模型。

Paper: LeCun et al., Gradient-Based Learning Applied to Document Recognition, Proc. IEEE, 1998

Chapter 4 | 训练要点

训练用浮点做，和硬件无关。关键是固定随机种子、归一化输入、记录 float 准确率，作为量化后的对比基线。

目标不是刷 SOTA：float 准确率超过 97%、int8 模拟后超过 95%，就足够说明整条链路是通的。

训练就像考试前的复习：给模型看很多带答案的题（MNIST 图片和标签），让它不断调整权重，把答错的题改正。

固定随机种子是为了“每次复习的起点一样”，这样实验结果可复现。float 准确率是量化前的成绩，int8 准确率是量化后的成绩，两份都要记录。

- 固定随机种子，保证可复现。
- MNIST 归一化到 $[0,1]$ 或 $[-1,1]$ 。
- Adam + 少量 epoch 即可收敛。
- 目标：float 准确率 $> 97\%$ ，int8 后 $> 95\%$ 。

Chapter 4 | 为什么要量化

- 硬件乘法器是 $\text{int8} \times \text{int8}$ 。
- float32 权重会占 4 倍存储。
- int8 推理是嵌入式 NPU 的事实标准。
- 量化误差可以通过 scale 逐层校正。

Chapter 4 | 后训练量化

后训练量化（PTQ）不需要重新训练：拿一批校准数据跑一遍，统计每层激活的 min/max，算出 scale；权重直接用自身的 min/max。

对称量化公式是 $q = \text{round}(\text{clamp}(x / \text{scale}, -128, 127))$ ， $\text{scale} = \max(|x|) / 127$ 。量化误差会累积，所以逐层用 int8 模拟验证很重要。

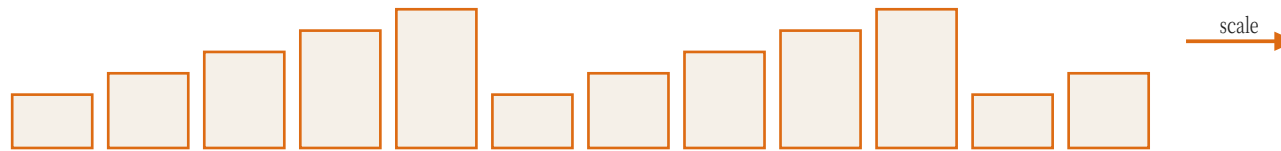
量化像把一张精细的照片压成 256 级灰度：先看整张照片最亮和最暗是多少（min/max），把亮度范围分成 256 格，每个像素落到最近的一格。

公式 $q = \text{round}(\text{clamp}(x/\text{scale}, -128, 127))$ 里的 clamp 是“夹住”：超出范围的数强行压到边界，round 是四舍五入。

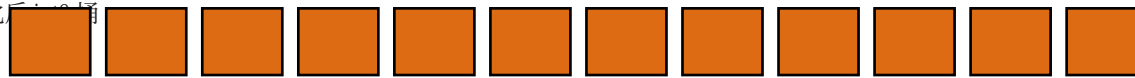
- 用校准集统计每层激活的 min/max。
- $\text{scale} = (\text{max} - \text{min}) / 255$ ，zero_point 可省略（对称量化）。
- $q = \text{round}(\text{clamp}(x / \text{scale}, -128, 127))$ 。
- 累加结果反量化： $\text{out_fp} = \text{out_int} * \text{scale_act} * \text{scale_w}$ 。

图 13：min/max 确定 scale，整个张量映射到 $[-128, 127]$ 。

浮点权重分布



量化后分布



Paper: Jacob et al., Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, CVPR 2018

Chapter 4 | BN 折叠

BatchNorm 在训练时依赖 batch 统计量，但推理时 mean/var 是常量，可以“折叠”进前一层的卷积权重： $w_{fold} = w * \gamma / \sqrt{\text{var} + \epsilon}$ ，bias 也一起合并。

折叠后 RTL 不需要实现 BN 单元，省掉一个模块，也少一层量化误差。

BatchNorm 像“先调音量再录音”：训练时它动态调，推理时音量固定了。固定之后，调音量可以和前一步录音合并成一步。

硬件里少一个模块就少一分出错可能，所以推理时把 BN 的缩放和偏移直接写进卷积权重，RTL 完全不用知道 BN 存在。

- BatchNorm 在推理时可以并入前一层卷积。
- $w_{fold} = w * \gamma / \sqrt{\text{var} + \epsilon}$ 。
- $b_{fold} = (b - \text{mean}) * \gamma / \sqrt{\text{var} + \epsilon} + \beta$ 。
- 这样 RTL 不需要单独实现 BN。

Chapter 4 | 导出格式

导出的内容有四种：权重（weights.hex）、一张测试图片（image.hex）、每层 golden 输出（golden.hex）、每层的 scale 和布局信息（scales.json）。

布局必须和 Ch3 的 SRAM 定义一致：行主序、int8 补码。这是软件和硬件之间最容易出错的地方。

导出文件就像快递单：weights.hex 是货物（权重），image.hex 是发货的图片，golden.hex 是签收标准（正确输出），scales.json 是尺寸说明（每层 scale）。

硬件和软件必须对同一张“快递单”有完全相同的理解：先放哪个数、每个数多宽、按什么顺序。这就是布局格式的重要性。

- weights.hex：逐层权重，行主序，每行一个 int8 补码十六进制。
- image.hex：一张测试图片。
- golden.hex：每层 int8 输出（或最终 logits）。
- scales.json：每层 scale / zero_point / 布局信息。



图 14：软件端先生成 golden，RTL 再逐层对齐。

Chapter 4 | Python int8 模拟器

“ Python int8 模拟器 ”就是在 PyTorch 里用整数运算重新实现前向传播：每个卷积/FC 输出先反量化再量化回 int8，激活也一样。

它跑出来的结果就是 RTL 的 golden。模拟器正确，RTL 才有对照；模拟器错了，RTL 再对也是错上加错。

在真正上硬件之前，先在 Python 里“模拟考”：用 int8 规则把模型完整跑一遍，得到标准答案。

这一步能帮你提前发现量化问题，比如某层误差特别大、scale 算错、布局不对。Python 模拟器对了，RTL 调试才有可靠的对照物。

- 用 PyTorch 张量实现量化 forward。
- 每层输出量化回 int8，与 RTL 行为一致。
- 这就是 RTL testbench 的 golden。
- 先把 Python 模拟器调对，再碰硬件。

Chapter 4 | 常见坑

最常见的是 round 规则不一致：Python 的 round() 是银行家舍入，硬件常用四舍五入，必须统一成同一种。

其次是 scale 精度：用 float 计算 scale 没问题，但导出后如果被截断成 int16，误差会被放大。还有 ReLU 之后 tensor 的 min 是 0，不能用负值范围。

Python 的 round() 是“银行家舍入”（0.5 可能舍成 0），硬件常用的四舍五入是“0.5 进 1”，两边必须统一。

另一个坑是 ReLU 后的激活全是非负数，min 不再是负数，如果还用训练前的 min/max 就会浪费一半量化范围。

- round 规则不一致（Python 银行家舍入 vs 四舍五入）。
- scale 用 float 计算但导出时精度丢失。
- ReLU 之后负半轴被截断，min 不能直接用负值。
- 权重/激活布局与 RTL 地址映射不一致。

Chapter 4 | 本章任务

- 训练 tiny CNN 并量化。
- 实现导出脚本与 Python int8 模拟器。
- 生成 npu_data/ 全部文件。
- 提交量化精度报告。

Chapter 4 | 总结

- 模型结构要和硬件能力对齐：小、int8、无复杂算子。
- 量化不是精度损失的结束，而是工程取舍的开始。
- 导出格式就是硬件和软件之间的接口。
- 下一章：把 NPU 挂回 Hack Computer，跑端到端推理。

术语速查 (Glossary)

- CPU：中央处理器，负责执行指令；在本课程里是 Hack CPU。
- RAM：随机存取存储器，Hack 有 64KB，用于放数据和程序状态。
- MMIO：Memory-Mapped I/O，把外设寄存器映射到内存地址，CPU 用普通读写指令控制外设。
- 寄存器：CPU 或外设内部的小容量存储单元，通常 8/16/32 位。
- SRAM：片上静态随机存储器，速度快、容量小，NPU 用它暂存权重和特征图。
- MAC：乘累加运算： $acc = acc + a * w$ ，是 CNN 最基础的计算。
- GEMM：通用矩阵乘， $C = A \times B$ ；卷积可以转换成 GEMM。
- PE：Processing Element，处理单元；本课程里是一个 MAC 单元。
- Systolic Array：脉动阵列：PE 排成网格，数据在相邻 PE 间逐拍流动。
- Weight-Stationary：权重驻留式数据流：权重装载后停在 PE 内反复使用。
- 斜输入：把第 row 行输入延迟 row 拍，使阵列各行在同一拍处理同一个 k 。
- im2col：Image to Column，把卷积窗口展平成矩阵行，从而用 GEMM 计算卷积。
- Line Buffer：行缓冲：只保留最近若干行，供滑动窗口复用。
- FSM：有限状态机：一组状态和跳转条件，NPU 控制器用它实现调度。
- DMA：Direct Memory Access，直接内存访问，批量搬运数据。

-
- 量化：把浮点数映射到低比特整数（如 int8），并记录 scale 以便还原。
 - Scale：量化比例因子，决定一个整数单位代表多大的浮点数值。
 - Polling：轮询：CPU 反复读状态寄存器，直到外设完成。
 - argmax：取最大值的下标；分类任务里就是预测类别。
 - NPU：Neural Processing Unit，神经网络处理器，专为张量运算设计的加速器。

References / 延伸阅读

- Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference", CVPR 2018.
- LeCun et al., "Gradient-Based Learning Applied to Document Recognition", Proceedings of the IEEE, 1998 (MNIST) .
- Han et al., "Deep Compression", ICLR 2016.
- Sze et al., "Efficient Processing of Deep Neural Networks", Proceedings of the IEEE, 2017.